IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 8 June 2025, accepted 12 August 2025, date of publication 20 August 2025, date of current version 26 August 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3600669

RESEARCH ARTICLE

GAT-APG: Graph Attention Network-Based Attack Path Generation for Security Simulation

MIN GEUN SONG, JAEWOONG CHOI, AND HUY KANG KIM, (Member, IEEE)
Graduate School of Information Security, Korea University, Seoul 02841, Republic of Korea
Corresponding author: Huy Kang Kim (cenda@korea.ac.kr)

This work was supported by a grant (UI247022TD) from the Agency for Defense Development, South Korea.

ABSTRACT Existing attack path generation methods face limitations in dynamic simulation environments due to their reliance on static network models and computational inefficiencies when network configurations change frequently. This study proposes GAT-APG, a reinforcement learning framework that combines Graph Attention Networks with policy gradient methods to generate adaptive attack paths for security simulation. The approach employs node-centric vulnerability assessment that transforms network traffic data into vulnerability metrics, enabling adaptation to network changes without complete graph recalculation. Experimental validation on controlled graphs and the Kyoto dataset demonstrates competitive performance, achieving 93% accuracy against brute-force methods and showing 69.1% exact matches with Dijkstra's algorithm on real-world topologies. The framework provides a simulation-ready environment for vulnerability assessment and defensive planning in dynamic networks.

INDEX TERMS Cyber attack simulation, graph neural network, reinforcement learning.

I. INTRODUCTION

As the integration of diverse networks accelerates, the complexity of system architectures continues to increase, leading to more sophisticated and intricate attacker behaviors [1], [2]. In these increasingly complex environments, even when human errors occur, detecting and correcting them becomes significantly more challenging. Attackers exploit such vulnerabilities, seeking to penetrate systems through unnoticed mistakes. Developing effective countermeasures has therefore become a critical challenge, driving research into proactive defense strategies. Initially, efforts focused on quantitatively evaluating network risks and designing mitigation measures through traditional risk management frameworks [3], [4]. More recently, research has expanded to include penetration testing, where defenders simulate attacker behavior, and probabilistic methods that generate and analyze attack paths and graphs. Given this progression, efficiently predicting attack surfaces and potential attack paths in advance, and applying appropriate countermeasures proactively, has become an urgent priority for securing modern complex systems.

However, existing defense methods have limitations in fully addressing the complexity and dynamic characteristics of modern network environments [5]. Traditional risk management approaches assume static network configurations, making it difficult to respond promptly to changes such as the addition of new nodes or modifications to access privileges [4]. Penetration testing techniques also often remain as one-time assessments conducted at a specific point in time, limiting their ability to reflect evolving threat landscapes [6]. Methods for generating and analyzing attack paths face similar challenges, as they typically require complete recomputation whenever network expansion or configuration changes occur, resulting in high computational complexity [7]. These limitations become particularly critical in large-scale environments that constantly evolve, such as critical national infrastructures and military communication systems [8]. In such complex and volatile environments, there is an increasing need for the ability to predict attack paths rapidly and continuously update defensive strategies [5]. Against this backdrop, a methodology that can respond sensitively to environmental changes and efficiently identify optimal

The associate editor coordinating the review of this manuscript and approving it for publication was Diana Gratiela Berbecaru.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

infiltration routes has become increasingly necessary. Reinforcement learning, which enables agents to autonomously learn optimal strategies through interaction with the environment, offers the potential to predict meaningful attack paths from the attacker's perspective even under dynamic network conditions [9]. In this study, we propose a method that leverages the characteristics of reinforcement learning to efficiently generate infiltration paths that attackers are likely to select in complex and dynamic network environments. Our approach aims to overcome the computational and adaptability limitations of existing defense methods, thus strengthening the ability to respond to threats in real time.

A. MOTIVATION

To address these challenges, reinforcement learning emerges as a promising paradigm. Reinforcement learning offers promising capabilities for autonomously learning optimal strategies through interactions with dynamic environments, making it a powerful tool for exploring potential attack paths in complex and evolving networks [9], [10]. However, several limitations arise when directly applying reinforcement learning to attack path generation. Most existing approaches rely on pre-constructed attack graphs or assume static network models, where the network topology and vulnerability information remain unchanged [4], [11]. Such assumptions severely limit their effectiveness in real-world settings, especially in sensitive and dynamic systems such as military communication networks or critical infrastructure [5]. In these environments, rapid and unpredictable changes occur frequently, and pre-built models quickly become outdated, hindering timely threat detection and response [12]. Furthermore, previous work has focused mainly on identifying potential attack paths without extending their scope to comprehensive attack simulations or attack-defense simulations [6], [8]. Modern security operations require not only the enumeration of the attack paths, but also a holistic simulation of how attacks unfold and how defenders might respond in real-time scenarios [13]. In critical systems, where failures can have widespread social impacts, it is essential to preemptively identify plausible intrusion scenarios and validate the effectiveness of defense strategies through systematic simulations [12], [14]. Another major shortcoming lies in the scalability and realism of existing models. Many approaches either overlook the complexity of real-world environments (*e.g.*, diverse device types, privilege hierarchies, and intricate vulnerability interactions) or suffer from computational inefficiencies that make large-scale simulations impractical [7], [10]. Consequently, these limitations reduce the applicability of existing methods for real-time security analysis and strategic defense preparation in critical environments. Given these challenges, this study aims to use reinforcement learning not only for enumeration of the attack path, but also for the complete simulation of the attacker's behaviors and the evaluation of defensive capabilities [9], [10]. Our goal is to develop an efficient and realistic attack/defense simulation model that can adapt to dynamic changes and scale effectively, particularly for applications in critical infrastructure protection and military systems. The motivations of this study can be summarized as follows:

- Methods that can generate and simulate attack scenarios in real time are needed, without relying on pre-constructed static graphs [4], [5].
- Beyond maximizing attack success rates, it is crucial to simulate attacker-defender interactions to evaluate realistic defense capabilities [6], [8].
- Simulation models must realistically reflect the complexity of real-world systems, especially in critical infrastructure environments [12], [14].
- To enable real-time based large-scale of simulations, the methods must minimize computational complexity while maintaining adaptability [7], [10].

Therefore, this study proposes a reinforcement learning-based attack/defense simulation framework that dynamically adapts to network changes, realistically models attacker behaviors, and evaluates defensive strategies in critical systems.

B. CONTRIBUTIONS

The main contributions of this work are:

- A reinforcement learning framework that combines Graph Attention Networks (GAT) with policy gradient methods for attack path generation in dynamic network environments, addressing computational and adaptability limitations of static graph-based approaches.
- A node-centric vulnerability assessment methodology that converts network traffic data into vulnerability scores for individual nodes, identifying vulnerability patterns from observed network behavior.
- An attack path generation system that adapts to network changes without requiring complete attack graph recalculation, supporting real-time applications in dynamic environments.
- Experimental validation using the Kyoto dataset, showing competitive performance with Dijkstra's algorithm (69.1% exact matches and 15.9% superior paths) and demonstrating applicability for network security assessment.
- A framework that supports vulnerability assessment, penetration testing automation, and security resource allocation by identifying likely attack paths.

The rest of this paper is organized as follows: Section II provides the necessary background on attack graphs, Bayesian attack graphs, vulnerability scoring systems, and graph neural networks. Section III reviews related work in rule-based, machine learning-based, Graph Neural Network (GNN)-based, and reinforcement learning-based approaches for attack path generation, along with attack-defense simulation frameworks. Section IV describes the Kyoto dataset used for experimental validation. Section IV presents a reinforcement learning-based attack path generation framework.

This framework is designed to reflect the vulnerability of each node while given traffic data is converted into the graphs; and the framework's machine learning model employs GAT-based architecture. Section VI details our experimental setup, testing methodology, and results on both controlled sample graphs and real-world network data. Finally, Section VII concludes the paper and discusses future research directions.

II. BACKGROUND

A. ATTACK GRAPH

An attack graph is a prominent tool for predicting network threats. It allows attackers to enumerate highly probable attack paths within system and network structures and visually represents them. By leveraging vulnerabilities and potential exploits, an attack graph deduces attack paths and enables risk assessment. Network topology, system information, and vulnerability databases like Common Vulnerabilities and Exposures (CVE) are primarily utilized to enumerate attack paths. The network topology provides information about the components and connectivity of devices in the network, while the system information includes detailed data about each component. The vulnerability database contains known vulnerabilities and patch information related to these vulnerabilities. Analyzing enumerated attack paths enables administrators to quickly identify assets that require protection measures and prioritize protection for those assets. By focusing on identified vulnerable areas in the system, administrators can apply protective measures more effectively, thus enhancing overall system stability.

Attack graphs are broadly classified by the modeling approach, the directionality of edges, and the presence of weights. Based on the modeling approach, attack graphs are classified into Bayesian and logical attack graphs. Bayesian attack graphs represent the probability of success of attacks from one node to another as weights. Logical attack graphs present the system components and attack paths logically. Additionally, considering the directionality of edges in attack graphs, they can be classified as directed graphs if edges have directionality or undirected graphs if edges do not. Furthermore, attack graphs can be categorized as weighted or non-weighted based on the presence or absence of weights. In this study, the Bayesian attack graph represents an undirected graph with weighted edges. It includes the shortest attack paths from the starting node to the remaining nodes, where the weight of each node is the weighted average of the Common Vulnerability Scoring System (CVSS) Score.

B. BAYESIAN ATTACK GRAPH (BAG)

Bayesian Attack Graphs (BAGs) is an attack graph modeling methodology that utilizes Bayesian networks to model potential attack paths [15]. The BAG model comprises nodes representing vulnerabilities and security elements within a system, each assigned conditional probabilities that reflect the impact of vulnerabilities and security elements on potential attacks. These conditional probabilities are represented in a Conditional Probability Table (CPT); they quantify the likelihood of an attacker succeeding based on vulnerabilities and security elements. Typically, the CVSS is employed to quantify the exploitability of vulnerabilities and security elements [3]. CVSS characterizes vulnerabilities and considers the probability that an attacker can successfully exploit them, providing an objective measure of their significance [16], which aids organizations in allocating security resources and managing vulnerabilities.

C. COMMON VULNERABILITIES AND EXPOSURES (CVE)

CVE is a publicly accessible identifier system for known security vulnerabilities managed by the National Vulnerability Database (NVD), a global vulnerability database [17]. It provides standardized methods for identifying and tracking vulnerabilities. CVE entries include vulnerability identifiers, vulnerability descriptions, affected software products, and version information impacted by the vulnerability. The CVE-ID follows the format CVE-YYYY-NNNN, where YYYY represents the year the vulnerability was discovered, and NNNN is a sequential number used to differentiate vulnerabilities discovered in that year. CVE aims to offer consistent information about vulnerabilities, making it easier to find and address specific vulnerabilities. It assists in quickly locating and responding to information about particular vulnerabilities.

D. COMMON VULNERABILITY SCORING SYSTEM (CVSS)

CVSS is a standardized framework used to assess and score the severity of vulnerabilities quantitatively [18]. Security experts use CVSS scores to evaluate the risk level of vulnerabilities and determine the priority of security measures. CVSS considers three aspects –technical aspects (Base Metrics), temporal aspects (Temporal Metrics), and environmental aspects (Environmental Metrics)– to score the severity of vulnerabilities. These three aspects are combined to calculate a score for the vulnerability, represented in a vector format. With CVSS scores, security experts can share information about the severity of vulnerabilities and use it as a reference for making decisions regarding security updates. CVSS base score can be divided into four severity levels based on the score: 0.1–3.9, 4.0–6.9, 7.0–8.9, 9.0–10.0 correspond to Low, Medium, High, and Critical levels, respectively.

E. GRAPH NEURAL NETWORK

GNN [19] is an artificial neural network that processes data expressed as a graph. Graph convolution converts the graph nodes into latent vectors (representation, embedding). Node features, edge features, and graph structure can affect the graph convolution. The calculated latent vector is used in various graph tasks such as node classification, edge classification, and graph classification. Numerous GNN-based neural models have been proposed for graph tasks.

The Graph Convolutional Network (GCN) is a GNN model that applies the idea of a Convolutional Neural Network (CNN) to graphs [20]. Unlike in Euclidean coordinate systems, graph structures are not fixed. Therefore, there are limitations in using CNN, which learns by moving filters, to represent graph structures. GCN generates node embeddings by aggregating information from neighboring nodes to represent each node.

GraphSAGE is a model designed to generate graph representations even on large-scale graphs efficiently [21]. When generating node embeddings, while GCN aggregates features from all neighboring nodes, this model samples neighboring nodes and uses only a subset of their features. Therefore, GraphSAGE is advantageous for representing graphs with scalability compared to GCN.

The GAT [22] is one of the numerous variations of GNN. GAT combines the attention mechanism with GNN. GAT performs graph convolution through self-attention based on the relationship between each node and its neighboring nodes. The significance of attention in GAT is to learn meaningful information from a node's neighbors, rather than the entire graph, when performing graph convolution for a specific node. The following formula can express the attention coefficient of GAT:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\vec{a}^T [\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_j]))}{\sum_{k\in\mathcal{N}_i} \exp(\text{LeakyReLU}(\vec{a}^T [\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_i]))}, \quad (1)$$

where $\mathbf{W}$ is the weight matrix, $\vec{h}_i$ is the feature of node i, $\mathcal{N}_i$ is the index set of node i and neighboring nodes, a is a weight vector, and $\parallel$ is a concatenation operation.

GAT also applied multi-head attention to stabilize the learning process of self-attention. The layers of GAT to which multi-head attention is applied are expressed as follows:

$$\vec{h}'_i = \Big\Vert_{k=1}^{K} \sigma\left(\sum_{j\in\mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \vec{h}_j\right), \quad (2)$$

where K is the number of heads, and σ is nonlinearity.

We used GAT to create a machine-learning model for generating attack graphs. The model receives network topology information in graph format as input and performs graph convolution to embed the network nodes. GAT is used for graph convolution.

III. RELATED WORK

Research in attack graphs and attack path generation focuses on modeling and visualizing attack sequences to enhance threat detection and response. Because attackers often employ a sequence of coordinated attacks within network systems to accomplish their goals. This body of work aims to efficiently generate attack paths, enabling timely responses to emerging security threats.

A. RULE-BASED MODELS FOR ATTACK PATH GENERATION

The rule-based model [23], [24], [25], [26] models attack scenarios based on network vulnerabilities and presents attack graph generation rules. Multi-host, multi-stage Vulnerability Analysis Language (MulVAL) [4] is an open-source framework designed to construct logical attack graphs [27]. Based on the proposed rule, an attack graph is generated, and security evaluation is performed. Stan et al. [14] presented a comprehensive network modeling that considers multiple network attack scenarios not supported by the existing MulVAL framework. Their research examined and modeled various network attack scenarios not covered by the current attack graph frameworks, conducting attack path and vulnerability analysis. Saha [28] proposed an algorithm that extends the MulVAL rules to include complex security policies and efficiently recalculates attack graphs to perform mutation analysis in response to network changes. Boudermine et al. [5] suggested an attack graph-based network security evaluation solution that considers changes in system properties. The inference rules in the proposed attack graph generation method modeled the cascading effects that can occur in the system. They defined the rules to reflect changes in network status over time. Binyamini et al. [29] proposed a transition probability matrix framework that can automatically generate new interaction rules by extracting attack entities based on word-embedding technology and text descriptions of the cyber attack database provided by NVD. The interaction rules generated by the framework can be integrated with the MulVAL attack graph tool. The proposed framework successfully automates time-consuming rule generation, but the success rate of interaction rule generation is 72%, which is low for commercialization, so there are imperfections. However, these MulVAL framework-based models have the limitation that they require recalculations whenever network changes occur, making them inefficient for networks that undergo frequent updates. Addressing these limitations is crucial. We need an automated update mechanism that can dynamically respond to frequent changes in dynamic networks. Equally important are optimized algorithms that can significantly enhance the efficiency of vulnerability-based threat analysis.

B. MACHINE LEARNING-BASED APPROACHES FOR ATTACK PATH GENERATION

Creating an attack graph with manual intervention from a security analyst is inefficient because it is time-consuming and labor-intensive. Some studies have applied machine learning models to automate attack graph generation to compensate for these shortcomings. Koo et al. [30] proposed a more lightweight approach to generate attack paths using attack graphs, aiming to improve the somewhat inefficient existing methodologies regarding cost and time. They restructured the attack graph generation problem as a multi-output and binary classification problem and provided a feature extraction method for training the attack graph generation model. Al Ghazo et al. [11] proposed automated attack graph generator and visualizer(A2G2V), an automated model-based attack graph generation and visualization algorithm.

However, the proposed model has limitations, as it involves tasks within the model, such as tool interaction and syntactic processing, which consume a considerable amount of time. Ghosh and Ghosh [7] designed an algorithm based on the variation of the AI search algorithm Planner, called SGPlan, to support automatic attack path generation within polynomial time, addressing scalability issues. They developed customized algorithms that are available as sub-level modules to enhance the time efficiency and scalability of the attack graph. The time complexity for attack path generation in this algorithm, with n hosts and m common vulnerabilities in the network, is $O(mn^3)$.

Bi et al. [31] proposed a model to overcome the limitations of complete attack graphs by generating attack graphs composed only of nodes containing essential information rather than generating all attack paths for all nodes. They presented a method that dynamically produces the top K attack paths based on the probability of attacking each node, resulting in attack paths instead of complete attack graphs. The complexity of the proposed method, when each node has M paths, and there are M nodes, is as follows:

$$O(K \cdot N^2 \cdot M \cdot \log(K \cdot N^2 \cdot M)). \tag{3}$$

Yu et al. [32] proposed an attack path generation model that can help with attack simulation and security analysis by addressing the issue of including duplicate nodes in the generated attack paths. They based their approach on the Breadth First Search (BFS) algorithm to find all non-redundant paths and used reverse search to remove unused nodes. While this method takes 30 seconds to generate attack paths, it becomes less efficient when applied to large-scale networks with high variability.

Jmal et al. [33] efficiently generated node embeddings for previously unseen data using the inductive property of GNN. They integrated network structural information as learnable features, enabling GNN-based approaches to adapt to the dynamic changes in network structures. Furthermore, they proposed an autonomous mitigation approach for high-risk attack paths by automatically adjusting the network's zero trust policy. This method achieved an average accuracy of over 95% and demonstrated a significant performance improvement by successfully identifying 47% more assets not detected by MulVAL.

C. GNN-BASED MODEL FOR ATTACK PATH GENERATION

Jmal et al. [33] proposed a GNN-based attack path identification method. The authors analyzed critical assets through risk assessment for attack path identification and adjusted network firewall rules and zero trust policies in a timely manner. Xu et al. [34] represented the web attack behavior of attackers using a graph data structure. The authors employed GAT, mapping nodes to web pages and edges to actions such as read, login, and search to represent attack patterns. However, there were limitations, such as ambiguity regarding whether specific actions were probing or actual attacks, and challenges in quickly issuing alerts. Rickli [35] emphasized the importance of learning path selection policy from attack graphs. The author utilized GAT to learn the path selection policy and generated attack graphs using GAT.

D. RL-BASED MODELS FOR ATTACK PATH GENERATION

Gao et al. [36] performed risk assessment based on Bayesian attack graphs using Q-learning Particle Swarm Optimization (QLPSO). The risk assessment targeted the Industrial Control System (ICS) environment, where CVSS was employed in the Bayesian attack graph as the attack success probability. Huang et al. [13] proposed a surveillance detection routes (SDR) method using reinforcement learning. The authors used various reinforcement learning techniques such as Advantage Actor-Critic (A2C), Double Agent Double agent Actor-Critic (DA-A2C), and Double agent Proximal Policy Optimization (DA-PPO). Gangupantulu et al. [8] proposed crown jewel analysis using reinforcement learning (CJA-RL). CJA-RL generates attack graphs using MulVAL, calculates the attack success rate based on CVSS, and computes the final attack path using RL. Wu et al. [37] derived attack paths using Fuzzy Petri-Net (FPN) and Q-learning. The authors divided the power Cyber-Physical System (CPS) network environment into information, sensing, and physical layers and conducted an attack path-based risk assessment based on the interactions between each layer. Cody [6] proposed a layered reference model for penetration testing, which includes generating attack graphs using Markov Decision Processs (MDP). However, the reference model is proposed without specific implementation details. Cody et al. [38] utilized MDP to generate attack graphs. The authors applied the CVSS score as a reward in reinforcement learning. Li et al. [39] proposed an reinforcement learning-based attack path prediction method in 5G industrial environments. The authors defined a dual network composed of a topology and vulnerability network; then, predicted attack paths using dual RL optimizing two objective functions. Yousefi et al. [10] proposed a Q-learning-based attack path generation method. The authors first created an attack graph using MulVUL and then derived possible attack paths from this graph using Q-learning. Nyberg et al. [9] trained cyber defense policies using reinforcement learning algorithms such as Proximal Policy Optimization (PPO) and Deep Q-Network (DQN) in a simulation environment based on attack graphs generated using Meta Attack Language (MAL).

E. METHODOLOGICAL CLASSIFICATION AND LIMITATIONS OF RELATED WORK

As illustrated in Table 1, research on attack graph generation can be broadly classified into four methods. Traditional rule-based methods require security experts to create rules, which are time-consuming during the manual regeneration process. Recent research has focused on various artificial intelligence-based methodologies to overcome these resource-intensive limitations. However, these AI-based approaches still face

challenges such as a high time complexity and complex preprocessing steps, which make them less straightforward to implement. In addition, specialized methodologies for specific domains may not be applied directly to Internet of Things (IoT) network environments. We propose an efficient attack path generation model from the perspective of a botnet attacker.

TABLE 1. Summary of methods in attack graph generation research.

Method	References
Rule-based	[14], [28], [5], [29]
ML-based	[7], [11], [30], [31], [32], [33]
GNN-based	[33], [34], [35]
reinforcement learning-based	[6], [8], [9], [10], [13], [36], [37], [38], [39]

F. ATTACK-DEFENSE SIMULATION WITH ATTACK PATH GENERATION

Recent research has increasingly emphasized the need for simulation frameworks that model both attacker and defender behaviors in dynamic network environments. These frameworks aim to provide adaptive responses to evolving threats and serve as testbeds for evaluating the effectiveness of cybersecurity strategies. Within this context, reinforcement learning has emerged as a promising approach to automate decision-making processes for both offensive and defensive agents.

Tang et al. [40] proposed a Hierarchical Multi-Agent Reinforcement Learning (HMARL) framework based on Stackelberg hypergame theory, in which attackers act as leaders and multiple defender agents operate collaboratively in a layered architecture. Their system enables timely policy regeneration in response to network state changes. However, the attacker strategies in their framework are based on predefined scenarios, limiting the adaptability of adversarial behavior generation.

Wu et al. [41] proposed a novel approach that integrates stochastic Petri nets with evolutionary game theory to model the dynamic interactions between attackers and defenders in network environments. Their model, termed the Stochastic Evolutionary Game Petri Net (SEGPN), captures the probabilistic and strategic behaviors of both parties, allowing for a more realistic simulation of attack-defense scenarios. By analyzing metrics such as attack success rates, average attack times, and system repair times, the SEGPN framework provides a quantitative assessment of system security under various strategies. This approach offers valuable insights into the temporal evolution of network security states, complementing reinforcement learning-based methods by providing a different perspective on modeling adaptive behaviors in cybersecurity contexts.

In parallel with formal modeling approaches such as Petri nets and game-theoretic frameworks, recent studies have increasingly adopted reinforcement learning to enhance the adaptability of agents within attack-defense simulation environments. For example, Nyberg et al. [9] developed a simulation system that employs PPO and DQN algorithms to train defensive agents in a graph-based network environment. Their framework incorporates CVSS scores and dynamic network topology to enable real-time adaptation to evolving threats. While their approach effectively models defender-side learning, it does not address the autonomous generation of attacker strategies.

Similarly, Huang et al. [13] proposed an attacker behavior simulation model using Adaptive Shortest Path Selection (ASPS) and actor-critic-based reinforcement learning algorithms (A2C, DA-PPO). Their model reconstructs attack paths in real-time within a multi-host network environment, particularly under military network constraints. Though the system provides realistic adversarial trajectories, it assumes a relatively fixed defensive structure and does not generalize to broader attack-defense co-evolution scenarios.

These frameworks highlight a trend toward modular, learning-driven attack-defense simulations that blend graph-based representations with strategic behavior modeling. Building on these developments, our study focuses on the design of an reinforcement learning-based attack path generation component that can be flexibly integrated into such simulation frameworks. Rather than constructing a complete simulation system, our objective is to contribute a learning-driven attack behavior module that is adaptable, scalable, and applicable to real-time scenarios such as military communication systems and critical infrastructure. Our approach supports the broader objective of proactive cybersecurity by enabling the simulation of realistic and adaptive threat trajectories.

IV. DATASET DESCRIPTION

The Kyoto dataset [42] represents a comprehensive benchmark for network security research, collected from real honeypot traffic at Kyoto University between 2006 and 2009. This dataset offers several critical advantages for attack path modeling research:

First, it provides exceptional temporal coverage and scale. The dataset captures network traffic data over a substantial period of approximately 3 years (November 2006 to August 2009), with data collection occurring on all but 36 days during this period (which were missed due to system maintenance and power failures). The dataset encompasses a massive volume of traffic: 93,076,270 total sessions, comprising 50,033,015 normal sessions, 42,617,536 known attack sessions, and 425,719 unknown attack sessions.

Second, it contains authentic attack patterns rather than simulated traffic. Unlike the KDD Cup 1999 dataset, which was generated through simulation, the Kyoto dataset contains actual traffic patterns including both normal and attack data. This authenticity is critical for developing models that can respond to real-world attack behaviors rather than idealized simulation scenarios.

Third, it provides rich feature diversity for a comprehensive analysis. It consists of 24 statistical features: 14 conventional

features derived from the KDD Cup 1999 dataset, plus 10 additional features designed for more effective analysis of modern cyber threats. The conventional features include basic traffic characteristics such as duration, service type, and byte counts, while the additional features incorporate data from Intrusion Detection System(IDS) alerts, malware detections, shellcode identifications, and session labels.

Fourth, it features multi-tool validation for attack identification. The traffic was thoroughly inspected using multiple security tools, including Symantec IDS, ClamAV antivirus, and the Ashula shellcode detector, which provides rich attack detection labels that increase confidence in attack classification accuracy.

Finally, it maintains privacy while preserving analytical value. To avoid any possible privacy concerns, all Internet Protocol (IP) addresses were sanitized using 'Unique Local IPv6 Unicast Addresses' while preserving the relationship between addresses within the same month.

These characteristics make the Kyoto dataset particularly valuable for our research on attack path modeling, as it provides both the authenticity and complexity needed to develop and validate realistic attack simulation models while maintaining sufficient structure for computational analysis.

V. PROPOSED METHOD

Figure 1 illustrates the overall architecture of our GAT-APG framework, which employs an encoder-decoder approach for adaptive attack path generation. The system processes the original network graph through a GAT-based encoder that captures node vulnerabilities and topological relationships via multi-head attention mechanisms. The encoded representations are then processed through multiple decoder layers to reconstruct graph features, followed by a reinforcement learning-based decoder that generates optimal attack paths through sequential decision-making. This three-stage architecture enables the framework to learn meaningful graph representations while adapting to network changes without requiring complete model retraining.

A. NETWORK NODE VULNERABILITY ASSESSMENT

Our vulnerability assessment methodology differs from traditional graph-based security modeling by focusing on individual node vulnerabilities rather than network topology. We calculate vulnerability scores for network nodes using actual traffic data features, similar to CVSS scoring principles but derived from observed network behaviors. This node-centric approach quantifies each node's security vulnerabilities based on traffic characteristics and behavioral patterns in the dataset. The methodology enables targeted security resource allocation and supports attack simulation by identifying nodes that attackers can prioritize based on vulnerability scores rather than solely topological considerations.

Our methodology implements a three-stage process for vulnerability quantification:

1) **Feature selection and extraction:** We utilized 17 of the 24 features available in the Kyoto dataset: "Source_IP_Address" and "Destination_IP_Address" helped identify unique nodes in the network, while the remaining 15 features contributed to determining individual vulnerability scores.
2) **Feature categorization and weighting:** We categorized these features into three groups with the following criteria:
   - Flags: Specific flag patterns observed in a node's traffic contribute to its vulnerability score, as certain combinations indicate potential security weaknesses. For example, SYN-ACK patterns without corresponding completion flags may indicate susceptibility to scanning or connection hijacking.
   - Service/Protocol: Services running on nodes receive vulnerability weights based on known security weaknesses, with higher weights assigned to services with documented vulnerabilities. This weighting system was developed through correlation analysis between service types and successful attack patterns within the dataset.
   - Other Features: Additional network behavior indicators are scaled and weighted based on their correlation with vulnerability using direct or inverse relationships depending on security implications. These relationships were established through statistical analysis of attack success rates in different feature values.
3) **Statistical consolidation:** For nodes with multiple traffics in the dataset, we applied statistical aggregation techniques: averaging for numerical features and mode calculation for categorical features to determine a consolidated vulnerability score for each unique node.

B. MODEL ARCHITECTURE

Our model employs a GAT architecture to find the optimal attack path based on reinforcement learning. This approach is inspired by the framework proposed by Drori et al. [43] to solve combinatorial optimization problems on graphs. GAT was selected due to its ability to dynamically assign importance weights to neighboring nodes, which is essential for simplifying graphs and identifying efficient optimal routes in heterogeneous networks.

1) ENCODER

The encoder consists of two graph attention layers followed by a feedforward layer. The first attention layer transforms the input features into higher-dimensional representations, while the second attention layer refines these representations. The model architecture can be described as follows:

$$\mathbf{h}_i^{(1)} = \text{Attention1}(\mathbf{x}, \mathbf{A}) \tag{4}$$

$$\mathbf{h}_i^{(2)} = \text{Attention2}(\mathbf{h}_i^{(1)}, \mathbf{A}) \tag{5}$$

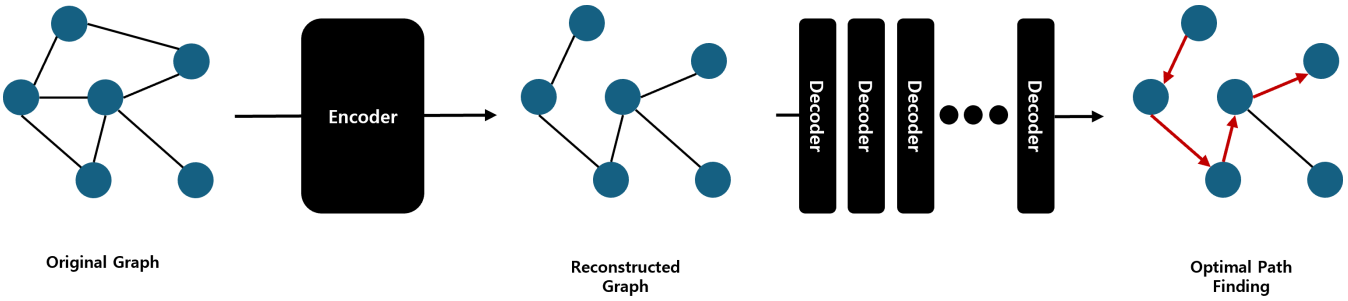


FIGURE 1. GAT-APG process flow.

$$\mathbf{h}_i^{(3)} = \text{FeedForward}(\mathbf{h}_i^{(2)}) \tag{6}$$
$$\mathbf{z}_i = \text{Softmax}(\mathbf{h}_i^{(3)}) \tag{7}$$

where $\mathbf{x}$ represents the node features, $\mathbf{A}$ is the adjacency matrix, and $\mathbf{z}_i$ is the final node representation.

The attention mechanism in each layer computes the attention coefficients between the nodes using a leaky ReLU activation function:

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_j]) \tag{8}$$
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k\in\mathcal{N}_i}\exp(e_{ik})} \tag{9}$$

where $\mathbf{W}$ is a learned weight matrix, $\mathbf{a}$ is the attention vector, and $\mathcal{N}_i$ represents the neighborhood of node i. Our implementation incorporates multi-head attention with 8 attention heads in the first layer and a single head in the second layer.

2) DECODER

The decoder determines the next node to visit based on the encoder's output and the current node. It implements a Depth-First Search (DFS) strategy enhanced with learned attention mechanisms. The attention coefficient computation in the decoder is given by:

$$a_{ij}^{dec} = C\tanh\left(\frac{(\Phi_1 v_i)^T(\Phi_2 v_j)}{\sqrt{d_h}}\right) \tag{10}$$

where C is a scaling constant set to 10, Φ_1 and Φ_2 are learnable transformations, v_i is the representation of the current node, v_j represents the candidate next nodes, and d_h is the dimensionality of the hidden representation.

The next node selection is determined probabilistically based on these attention coefficients, with the highest coefficient node being selected:

$$p_i = \text{Softmax}(a_i^{dec} \odot \mathbf{m}) \tag{11}$$
$$\text{next_node} = \arg\max(p_i) \tag{12}$$

where $\mathbf{m}$ is a mask that ensures only unvisited neighbors are considered, and $\odot$ represents element-wise multiplication.

3) PATH GENERATION ALGORITHM

The complete path generation process follows a modified depth-first search algorithm. Our path generation approach maintains a visited nodes list starting with a designated start node, and systematically explores the graph while tracking branch points for potential backtracking.

For each step, the algorithm first identifies all valid neighboring nodes that have not been visited yet. If multiple unvisited neighbors exist, the current node is marked as a branch point for potential backtracking later. When the algorithm reaches a node with no unvisited neighbors, it backtracks to the most recent branch point and continues exploration from there.

The decoder component selects the next node using learned attention mechanisms, considering only valid neighbors. As the path grows, the algorithm accumulates vulnerability weights and calculates rewards based on the cumulative vulnerability score. This approach ensures thorough exploration of the network while prioritizing paths with higher vulnerability scores.

Throughout this process, the accumulated weights and rewards track the vulnerability score of the path, with higher values indicating more attractive attack paths from an attacker's perspective.

4) REINFORCEMENT LEARNING COMPONENT

The reinforcement learning aspect of our model is implemented using a policy gradient approach [43]. The reward function and the loss calculation are defined as follows:

$$r = \sum_{i=1}^{|V|}\left(\sum_{e\in\mathcal{Q}_i} W(e)\right) \tag{13}$$
$$\mathcal{L}(\theta) = \sum_{t=1}^{n}(r_t - b)\log p\,(a_t) \tag{14}$$

where r_t is the reward at step t, b is a baseline value, and $p(a_t)$ is the probability of selecting the action a_t. The model is trained to maximize the cumulative reward, which corresponds to finding paths with high vulnerability scores.

The baseline b helps reduce variance in the policy gradient estimation, while the log-probability term $\log p(a_t)$ guides the model toward actions that yield higher-than-baseline

rewards and away from actions that yield lower-than-baseline rewards.

VI. EXPERIMENTS

A. TRAINING

The model was trained on randomly generated graphs to show generalizability across different network topologies. The training dataset consisted of undirected weighted graphs with node counts ranging from 20 to 200 nodes.

During training, we used the Adam optimizer with a learning rate of 0.0005 and a dropout rate of 0.5. The GAT model was configured with 8 attention heads, 128 hidden features, and trained for 100 epochs. We selected hyperparameters that yielded the highest reward values during validation.

The training process focused on optimizing the model's ability to identify paths with maximum reward, which corresponds to the cumulative vulnerability weights.

B. TESTING METHODOLOGY AND EXPERIMENTAL DESIGN

We evaluated our model using two distinct testing approaches: controlled sample graphs to establish baseline performance metrics and a realistic network derived from the Kyoto dataset to validate real-world applicability. This dual validation approach allows us to both precisely measure model accuracy under controlled conditions and assess its practical effectiveness in complex real-world network scenarios.

1) SAMPLE GRAPH TESTING

The controlled graph experiments were designed to systematically evaluate how model performance varies with increasing network complexity, while maintaining controlled variables that allow precise performance measurement. For our controlled experiments, we constructed sample graphs with the following characteristics:

- Fixed node count: 100 nodes in all test graphs, ensuring that performance differences were attributable to connectivity rather than network size
- Variable edge counts: 110, 120, 130, and 140 edges, representing increasingly complex network connectivity patterns
- Consistent node vulnerability weights distribution, eliminating variability in vulnerability distributions as a confounding factor

To establish ground truth for these sample graphs, we utilized a brute-force algorithm that exhaustively checks all possible paths, ensuring that we had the true optimal solutions for comparison. While computational complexity prevented brute-force validation on larger graphs, these controlled experiments with 100 nodes represent a significant network size that still allows for exhaustive validation. The brute-force algorithm implemented a depth-first search with branch pruning to identify paths with maximum cumulative vulnerability scores, providing the absolute optimal paths against which our model's output could be compared. This comprehensive approach, while computationally intensive, provided an absolute benchmark against which to measure our model's accuracy.

TABLE 2. Performance for graphs with different edge densities.

Number of Edges	Accuracy (%)	Reward Difference	Mean Reward Difference
110	93.00	2.7134	0.0271
120	81.00	8.2509	0.0825
130	64.00	20.8032	0.2080
140	63.00	26.3848	0.2638

2) KYOTO DATASET TESTING

For validating real-world performance, we required a testing approach that could accommodate the scale and complexity of actual network traffic data while still providing a meaningful performance baseline. For real-world validation, we implemented a four-step testing process using the Kyoto dataset:

1) Creating a network topology graph from the Kyoto traffic data, translating communication patterns between IP addresses into a network graph structure
2) Assigning vulnerability weights to nodes based on traffic features, using our vulnerability scoring framework to quantify node-specific risk levels
3) Running our trained GAT model to generate attack paths, identifying likely paths an attacker would take through the network
4) Comparing the results against paths generated by Dijkstra's algorithm, providing a systematic performance baseline

For this larger, more complex network, we employed Dijkstra's algorithm (modified to maximize rather than minimize weights) as our comparison baseline instead of brute-force methods, which would be computationally infeasible at this scale. Dijkstra's algorithm provides an appropriate baseline because it represents the optimal path finding approach under the assumption that attackers always choose the mathematically optimal path. While actual attackers may not always behave optimally, this comparison allows us to measure how closely our model approximates mathematical optimality while incorporating the learning-based advantages of our GAT architecture. The modified Dijkstra implementation used identical node vulnerability weights to our model, ensuring a fair comparison focusing on path-finding capability rather than vulnerability assessment.

C. RESULTS

The experimental results demonstrate our model's performance characteristics across both testing scenarios.

1) SAMPLE GRAPH TESTING RESULTS

The accuracy of our model on sample graphs showed a clear correlation with graph complexity, as presented in Table 2 and Figure 2.

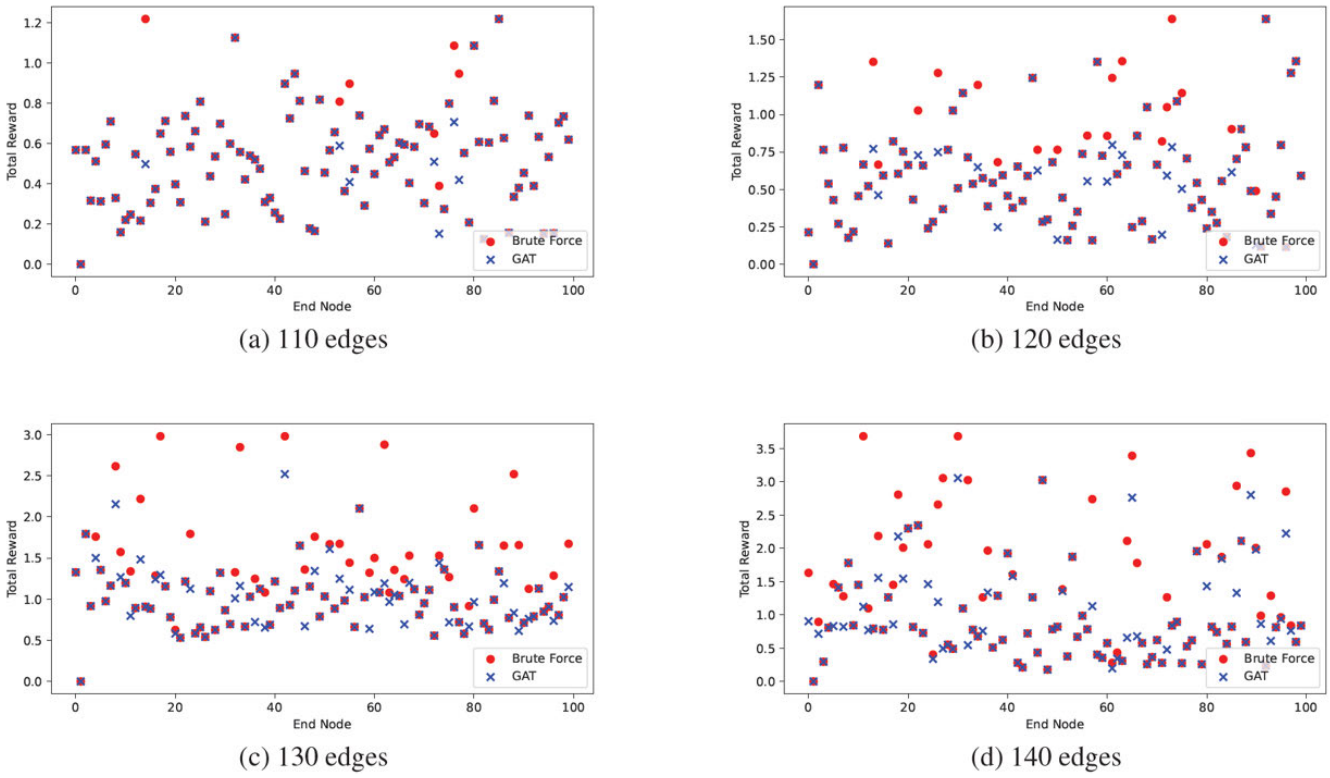


FIGURE 2. Model performance comparison against brute-force baseline on 100-node graphs with varying edge densities.

The sample graph results demonstrate a significant inverse relationship between graph complexity and model accuracy. With 110 edges, our model achieves 93% accuracy compared to the optimal brute-force solution, indicating strong performance in identifying vulnerable attack paths in moderately complex networks. However, this accuracy decreases substantially as the edge density increases, dropping to 63% with 140 edges. Notably, while performance degradation is consistent, the rate of degradation diminishes as edge density increases: the accuracy drops by 12% (from 93% to 81%) between 110 and 120 edges, then by 17% (from 81% to 64%) between 120 and 130 edges, and only by 1% (from 64% to 63%) between 130 and 140 edges.

The reward difference analysis provides additional insights into model performance degradation compared to the brute-force baseline. The total reward difference between our GAT model and the brute-force optimal solution increases from 2.7134 with 110 edges to 26.3848 with 140 edges, representing a nearly ten-fold increase in the performance gap with the brute-force baseline. The mean reward difference per test case similarly escalates from 0.0271 to 0.2638, indicating that individual path quality deteriorates relative to the brute-force optimum as network complexity increases.

This degradation pattern suggests that, as simulation networks become more interconnected, the model faces increasing challenges in distinguishing optimal paths from near-optimal alternatives. The attention mechanism in GAT, while effective in moderately connected graphs, appears to struggle with the increased number of competing paths in highly dense networks. Figure 2 illustrates this trend across all four edge density configurations, showing a consistent performance decline as connectivity increases.

2) KYOTO DATASET TESTING RESULTS

The comparative analysis between our GAT model and the Dijkstra algorithm revealed competitive performance in real-world network scenarios. When tested on 1115 nodes from the Kyoto dataset network topology, the mean reward difference between approaches was remarkably small at 0.019110, with 771 nodes (69.1%) showing exact matches. The GAT model identified superior paths for 177 nodes (15.9%), while Dijkstra found better paths for 167 nodes (15.0%), indicating nearly balanced performance. The minimum difference reward of -46.315200 showed cases where our model substantially outperformed Dijkstra, while the maximum positive difference reward of 43.635844 represented instances where Dijkstra discovered better paths.

A key advantage of our approach is its ability to reconstruct and simplify network topologies by identifying critical attack paths. Figure 3 illustrates this capability using the Kyoto dataset, where black edges represent connections retained as critical for attack paths, and red edges indicate connections eliminated as non-essential. This reconstruction process enables network simplification while preserving

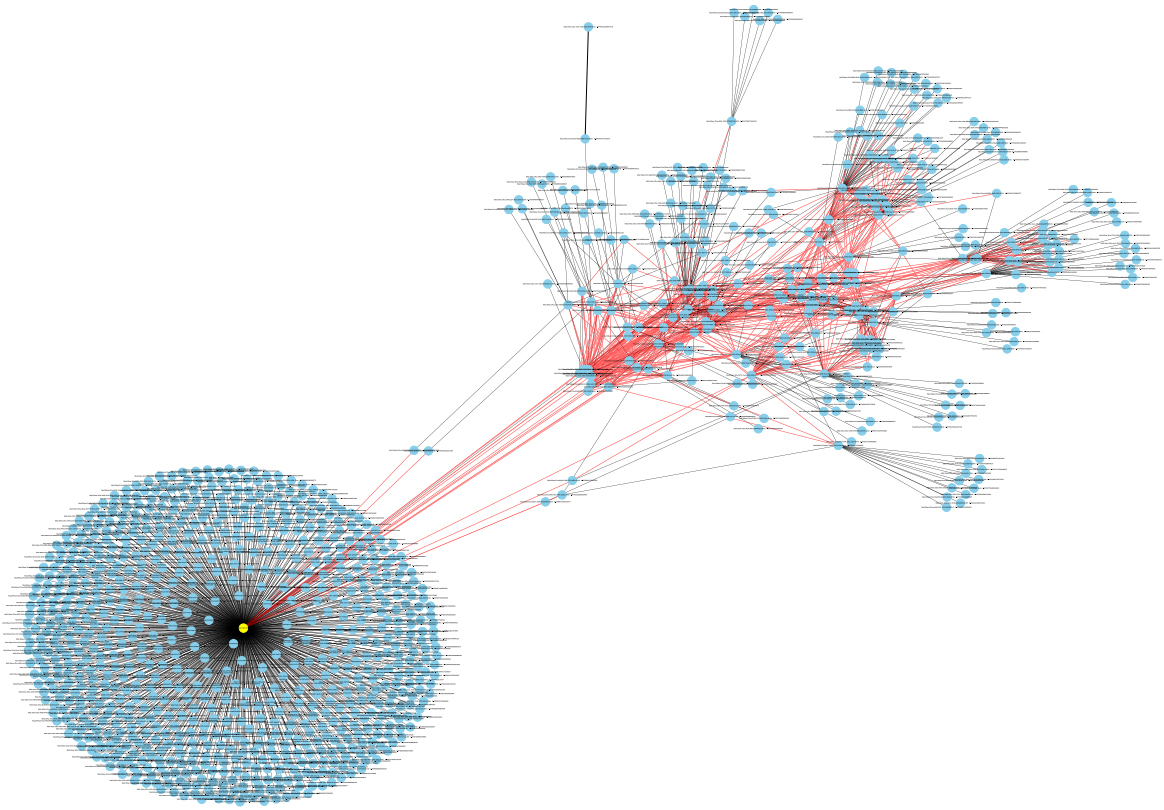


FIGURE 3. Original vs. reconstructed graph: retained (black) and eliminated (red) edges (kyoto dataset).

important attack vectors, allowing security teams to focus resources on the most likely attack pathways. The complete network visualizations before and after reconstruction are shown in Appendix, demonstrating how our model provides actionable intelligence for network topology optimization beyond simple path finding.

VII. CONCLUSION

This paper presents a reinforcement learning-based approach for efficiently identifying potential attack paths in dynamic network environments, with a specific focus on simulation-based communication networks. By leveraging GAT and reinforcement learning techniques, our model provides an effective solution to generate attack paths that attackers are likely to exploit, enabling proactive security measures in complex and evolving network landscapes.

Our experimental results demonstrate the viability of our approach across both controlled sample graphs and real-world network data derived from the Kyoto dataset. In sample graph testing, our model achieved up to 90% accuracy in identifying optimal attack paths compared to brute-force methods, although this accuracy decreased to 63% as network complexity increased. This indicates that the model performs most effectively in networks with moderate connectivity rather than in highly dense configurations, which aligns with typical simulation scenarios where communication density varies based on different operational parameters.

When tested against the Kyoto dataset using a more practical Dijkstra algorithm as the baseline, our GAT model showed competitive performance, with a near-zero mean difference (0.019110) in path quality. Notably, our model discovered superior paths in 15.9% of test cases, while matching Dijkstra's results exactly in 69.1% of cases. This demonstrates that our approach can effectively identify high-vulnerability paths without the computational burden associated with traditional graph algorithms, making it particularly suitable for real-time security monitoring in dynamic simulation networks.

A. APPLICABILITY IN SIMULATION ENVIRONMENTS

Our approach has particularly valuable applications in simulation environments for identifying optimal attack routes from an attacker's perspective. As demonstrated in our experimental results, the model effectively identifies vulnerable attack paths in simulated network environments. Specifically, as shown in our visualization comparing original and reconstructed graphs, our model possesses the capability to identify and remove non-critical connections (red edges) from the original graph, allowing focus on paths that attackers are most likely to select in real-world scenarios.

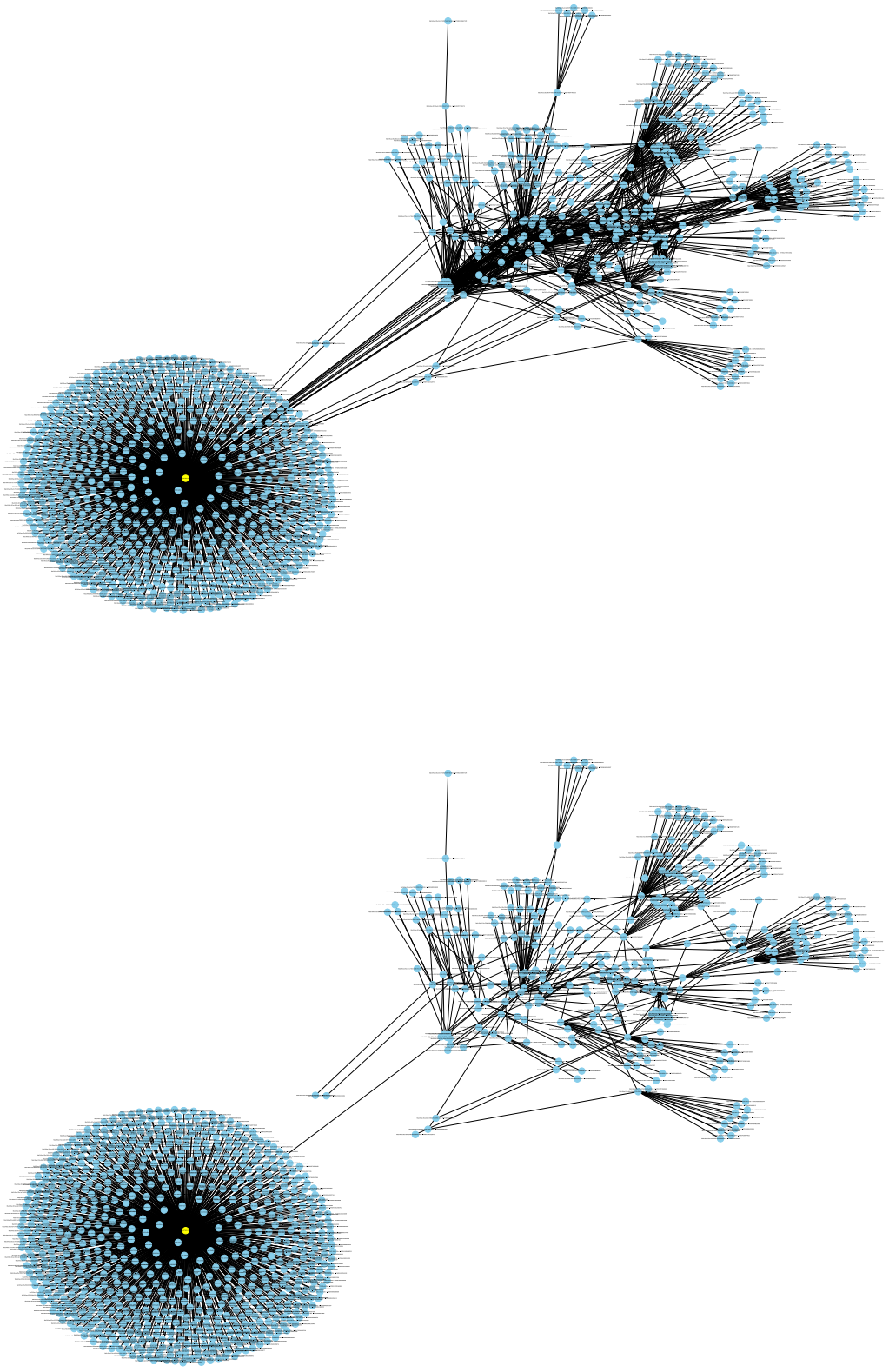


FIGURE 4. Original (top) and reconstructed (bottom) graph visualizations (Kyoto dataset).

This simulation capability provides several practical benefits to security professionals. Pre-deployment vulnerability assessment allows security teams to simulate likely attack paths before deploying network configurations, identifying

and addressing critical vulnerabilities proactively. Red-team automation enables the model to automate aspects of red-team exercises by identifying non-obvious attack paths that might be overlooked in manual penetration testing. Defensive resource optimization becomes possible by identifying the most likely attack paths, enabling more efficient allocation of monitoring and defensive resources to critical network segments.

This capability is implemented through our path generation algorithm and the reinforcement learning component, enabling realistic simulation of attacker decision-making processes across various network configurations. This provides defenders with the opportunity to anticipate attacker behavior and proactively address vulnerabilities before they can be exploited.

The key advantage of our approach is its ability to adapt to network changes without requiring complete recalculation of attack graphs. Our pre-trained model trained on diverse graph topologies addresses the fundamental limitation of existing attack path generation methods that struggle with dynamic simulation environments. Unlike traditional methods that require rebuilding entire models when network configurations change, our pre-trained GAT model can generalize to new network topologies without retraining, making it inherently suitable for dynamic environments. By transforming traditional traffic data into node-specific vulnerability metrics, our model enables targeted security hardening of the most vulnerable components rather than applying uniform security measures across the network. This is especially valuable in simulation environments where the network topology may change frequently due to dynamic scenarios and varying communication conditions.

B. LIMITATIONS AND FUTURE WORK

Despite these advances, our research has several limitations that warrant consideration. Performance degradation in highly complex networks becomes evident, as shown in our experimental results, where accuracy decreases significantly as network complexity increases, indicating challenges in handling extremely dense network configurations. The dependency on representative training data poses another limitation, as the model's effectiveness depends on training data that accurately represent potential attack behaviors, which may be difficult to obtain for novel attack types. Limited temporal dynamics present additional challenges, as the current implementation does not fully account for the temporal aspects of attacks that evolve over time, potentially missing sophisticated multi-stage attack scenarios. The computational requirements for initial training represent a practical constraint, where while inference is efficient, the initial training process requires significant computational resources that may not be available in all security environments.

Future work should focus on enhancing the model's performance in highly complex networks, incorporating temporal dynamics of attacks, and integrating defender perspective simulations to create a comprehensive attack-defense framework. Furthermore, testing the model in actual deployment scenarios would provide valuable insight into its practical efficacy in critical infrastructure protection applications.

In conclusion, our reinforcement learning-based attack path generation approach offers a promising solution for improving network security in dynamic simulation environments by efficiently identifying potential vulnerabilities and enabling proactive defense strategies with reduced computational overhead.

APPENDIX
GRAPH RECONSTRUCTION

See Figure 4.

ACKNOWLEDGMENT

This work was supported by a grant (UI247022TD) from the Agency for Defense Development, South Korea.

REFERENCES

[1] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.

[2] H. Zhao, H. Shu, and Y. Xing, "A review on IoT botnet," in *Proc. 2nd Int. Conf. Comput. Data Sci.*, Jan. 2021, pp. 1–7.

[3] M. Frigault, L. Wang, S. Jajodia, and A. Singhal, "Measuring the overall network security by combining CVSS scores based on attack graphs and Bayesian networks," *Netw. Secur. Metrics*, pp. 1–23, Jun. 2017.

[4] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. USENIX Secur. Symp.*, 2005, p. 8.

[5] A. Boudermine, R. Khatoun, and J.-H. Choyer, "Attack graph-based solution for vulnerabilities impact assessment in dynamic environment," in *Proc. 5th Conf. Cloud Internet Things (CIoT)*, Mar. 2022, pp. 24–31.

[6] T. Cody, "A layered reference model for penetration testing with reinforcement learning and attack graphs," in *Proc. IEEE 29th Annu. Softw. Technol. Conf. (STC)*, Oct. 2022, pp. 41–50.

[7] N. Ghosh and S. K. Ghosh, "A planner-based approach to generate and analyze minimal attack graph," *Int. J. Speech Technol.*, vol. 36, no. 2, pp. 369–390, Mar. 2012.

[8] R. Gangupantulu, T. Cody, A. Rahma, C. Redino, R. Clark, and P. Park, "Crown jewels analysis using reinforcement learning with attack graphs," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2021, pp. 1–6.

[9] J. Nyberg, P. Johnson, and A. Méhes, "Cyber threat response using reinforcement learning in graph-based attack simulations," in *Proc. IEEE/IFIP Netw. Oper. Manage. Symp. (NOMS)*, Apr. 2022, pp. 1–4.

[10] M. Yousefi, N. Mtetwa, Y. Zhang, and H. Tianfield, "A reinforcement learning approach for attack graph analysis," in *Proc. 17th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun./ 12th IEEE Int. Conf. Big Data Sci. Eng. (TrustCom/BigDataSE)*, Aug. 2018, pp. 212–217.

[11] A. T. A. Ghazo, M. Ibrahim, H. Ren, and R. Kumar, "A2G2V: Automatic attack graph generation and visualization and its applications to computer and SCADA networks," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 50, no. 10, pp. 3488–3498, Oct. 2020.

[12] S. Hou, X. Chen, J. Ma, Z. Zhou, and H. Yu, "An ontology-based dynamic attack graph generation approach for the Internet of Vehicles," *Frontiers Energy Res.*, vol. 10, Jun. 2022, Art. no. 928919.

[13] L. Huang, T. Cody, C. Redino, A. Rahman, A. Kakkar, D. Kushwaha, C. Wang, R. Clark, D. Radke, P. Beling, and E. Bowen, "Exposing surveillance detection routes via reinforcement learning, attack graphs, and cyber terrain," in *Proc. 21st IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2022, pp. 1350–1357.

[14] O. Stan, R. Bitton, M. Ezrets, M. Dadon, M. Inokuchi, Y. Ohta, T. Yagyu, Y. Elovici, and A. Shabtai, "Extending attack graphs to represent cyber-attacks in communication protocols and modern IT networks," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1936–1954, May 2022.

[15] Y. Liu and H. Man, "Network vulnerability assessment using Bayesian networks," *Proc. SPIE*, vol. 5812, pp. 61–71, Apr. 2005.

[16] A. Sahu and K. Davis, "Structural learning techniques for Bayesian attack graphs in cyber physical power systems," in *Proc. IEEE Texas Power Energy Conf. (TPEC)*, Feb. 2021, pp. 1–6.
[17] MITRE Corp. (Sep. 21, 2023). *CVE*. [Online]. Available: https://cve.mitre.org/
[18] P. Mell, K. Scarfone, and S. Romanosky, "Common vulnerability scoring system," *IEEE Secur. Privacy Mag.*, vol. 4, no. 6, pp. 85–89, Nov. 2006.
[19] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2008.
[20] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
[21] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, p. 1.
[22] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lió, and Y. Bengio, "Graph attention networks," *Stat*, vol. 1050, no. 20, Feb. 2017, Art. no. 1048550.
[23] C. Hankin and P. Malacaria, "Attack dynamics: An automatic attack graph generation framework based on system topology, CAPEC, CWE, and CVE databases," *Comput. Secur.*, vol. 123, Dec. 2022, Art. no. 102938.
[24] Y. Feng, G. Sun, Z. Liu, C. Wu, X. Zhu, Z. Wang, and B. Wang, "Attack graph generation and visualization for industrial control network," in *Proc. 39th Chin. Control Conf. (CCC)*, Jul. 2020, pp. 7655–7660.
[25] A. Xie, L. Zhang, J. Hu, and Z. Chen, "A probability-based approach to attack graphs generation," in *Proc. 2nd Int. Symp. Electron. Commerce Secur.*, vol. 2, May 2009, pp. 343–347.
[26] S. Zhong, D. Yan, and C. Liu, "Automatic generation of host-based network attack graph," in *Proc. WRI World Congr. Comput. Sci. Inf. Eng.*, vol. 1, Mar. 2009, pp. 93–98.
[27] D. Tayouri, N. Baum, A. Shabtai, and R. Puzis, "A survey of MulVAL extensions and their attack scenarios coverage," *IEEE Access*, vol. 11, pp. 27974–27991, 2023.
[28] D. Saha, "Extending logical attack graphs for efficient vulnerability analysis," in *Proc. 15th ACM Conf. Comput. Commun. Secur.*, Oct. 2008, pp. 63–74.
[29] H. Binyamini, R. Bitton, M. Inokuchi, T. Yagyu, Y. Elovici, and A. Shabtai, "A framework for modeling cyber attack techniques from security vulnerability descriptions," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 2574–2583.
[30] K. Koo, D. Moon, J.-H. Huh, S.-H. Jung, and H. Lee, "Attack graph generation with machine learning for network security," *Electronics*, vol. 11, no. 9, p. 1332, Apr. 2022.
[31] K. Bi, D. Han, and J. Wang, "K maximum probability attack paths dynamic generation algorithm," *Comput. Sci. Inf. Syst.*, vol. 13, no. 2, pp. 677–689, 2016.
[32] X. Yu, J. Jiang, and C. Shuai, "Approach to attack path generation based on vulnerability correlation," in *Proc. IEEE Conf. Anthology*, Jan. 2013, pp. 1–6.
[33] H. Jmal, F. Ben Hmida, N. Basta, M. Ikram, M. A. Kaafar, and A. Walker, "SPGNN-API: A transferable graph neural network for attack paths identification and autonomous mitigation," 2023, *arXiv:2305.19487*.
[34] Y. Xu, Y. Fang, Z. Liu, and Q. Zhang, "PWAGAT: Potential web attacker detection based on graph attention network," *Neurocomputing*, vol. 557, Nov. 2023, Art. no. 126725.
[35] M. Rickli, "Learning policies for path selection in attack graphs," KTH Royal Inst. Technol., Stockholm, Sweden, Tech. Rep. TRITA-EECS-EX-2022:38, 2022.
[36] X. Gao, Y. Zhou, L. Xu, and D. Zhao, "Optimal security protection strategy selection model based on Q-learning particle swarm optimization," *Entropy*, vol. 24, no. 12, p. 1727, Nov. 2022.
[37] R. Wu, J. Gong, W. Tong, and B. Fan, "Network attack path selection and evaluation based on Q-learning," *Appl. Sci.*, vol. 11, no. 1, p. 285, Dec. 2020.
[38] T. Cody, A. Rahman, C. Redino, L. Huang, R. Clark, A. Kakkar, D. Kushwaha, P. Park, P. Beling, and E. Bowen, "Discovering exfiltration paths using reinforcement learning with attack graphs," in *Proc. IEEE Conf. Dependable Secure Comput. (DSC)*, Jun. 2022, pp. 1–8.
[39] X. Li, X. Hu, and T. Jiang, "Dual-reinforcement-learning-based attack path prediction for 5G industrial cyber-physical systems," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 50–58, Jan. 2024.
[40] Y. Tang, J. Sun, H. Wang, J. Deng, L. Tong, and W. Xu, "A method of network attack-defense game and collaborative defense decision-making based on hierarchical multi-agent reinforcement learning," *Comput. Secur.*, vol. 142, Jul. 2024, Art. no. 103871.
[41] Z. Wu, L. Tian, Y. Zhang, Y. Wang, and Y. Du, "Network attack and defense modeling and system security analysis: A novel approach using stochastic evolutionary game Petri net," *Secur. Commun. Netw.*, vol. 2021, pp. 1–10, Nov. 2021.
[42] J. Song, H. Takakura, and Y. Okabe. (2006). *Description of Kyoto University Benchmark Data*. Accessed: Mar. 15, 2016. [Online]. Available: http://www.takakura.com/Kyoto_data/BenchmarkData-Description-v5
[43] I. Drori, A. Kharkar, W. R. Sickinger, B. Kates, Q. Ma, S. Ge, E. Dolev, B. Dietrich, D. P. Williamson, and M. Udell, "Learning to solve combinatorial optimization problems on real-world graphs in linear time," in *Proc. 19th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2020, pp. 19–24.



MIN GEUN SONG received the B.S. degree in statistics and information security from Korea University, Seoul, South Korea, in 2023. He is currently pursuing the integrated M.S. and Ph.D. degree in information security. His research interests include artificial intelligence and attack graphs.



JAEWOONG CHOI received the B.S. degree in cybersecurity from Pai Chai University, Daejeon, Republic of Korea, in 2022. He is currently pursuing the integrated M.S. and Ph.D. degree in information security with the Graduate School of Information Security, Korea University, Seoul, Republic of Korea. His research interests include data analysis, online game security, and social engineering analysis.



HUY KANG KIM (Member, IEEE) received the B.S. degree in industrial management, the M.S. degree in industrial engineering, and the Ph.D. degree in industrial and systems engineering from Korea Advanced Institute of Science and Technology (KAIST), in 1998, 2000, and 2009, respectively. He founded A3 Security Consulting, South Korea, in 1999, the first information security consulting company. Currently, he is a Professor with the School of Cybersecurity, Korea University. Before joining Korea University, he was the Technical Director (TD) and the Head of the Information Security Department, NCSOFT, from 2004 to 2010, one of the most famous MMORPG companies in the world. His current research interest includes solving many security problems in online games based on user behavior analysis. He was a member and the last Leader of the KAIST UNIX Society (KUS), South Korea, the legendary hacking group.